Téma 10: Základní principy testování

Teoretický základ

Neotestovaný kód je rozbitý kód. Testy (konkrétně Unit testy) ověřují izolovaně nejmenší funkční celky (typicky metody a funkce).

- **Klíčové slovo `assert`:** Zkontroluje, zda je podmínka pravdivá. Pokud ne, program ihned tvrdě spadne (AssertionError). Zabráníme tak pokračování programu s chybnými daty.
- **Modul `unittest`:** Robustní framework přímo vestavěný v Pythonu pro psaní organizovaných testů. Testovací třída vždy dědí z `unittest.TestCase` a každá testovací metoda musí začínat slovem `test_`.
- Využívá se testovacích metod jako `assertEqual(a, b)` nebo `assertTrue(x)`.

Základní koncept k zamyšlení: Test-Driven Development (TDD)

Metodika TDD (Testy Řízený Vývoj) přistupuje k programování obráceně. Testy se dle ní píší dříve, než samotný aplikační kód. Programátor si tím dokonale nadefinuje očekávané chování. Napsaný test napřed samozřejmě selže (tzv. Red fáze), následně se napíše produkční kód, který jej splní (Green fáze) a nakonec se vše učesává a zpřehledňuje (Refactor).

Úkol 1 (Lehká obtížnost)

Zadání úkolu: Napište kratičkou funkci pro sčítání čísel. Hned na další řádek pod ní si napište jednoduchý a bleskový řádkový test: použijte klíčové slovo `assert`, abyste ověřili, že volání funkce pro čísla 2 a 3 vrátí opravdu výsledek 5. Přidejte i chybovou zprávu pro případ selhání.

Řešení úkolu 1

```
def secti(a, b):
    return a + b

# Zkouška izolované funkčnosti (nejjednodušší možný typ Unit Testu)
# Pokud je výraz roven True, program pokračuje tiše bez povšimnutí.
# Kdyby byla funkce zapsána špatně, assert zastaví program tvrdou chybou.
assert secti(2, 3) == 5, "Systémová chyba: sčítání nefunguje správně!"
```

Úkol 2 (Střední obtížnost)

Zadání úkolu: Vytvořte pokročilejší zkoušku pro funkci odčítání pomocí vestavěné vývojářské knihovny `unittest`. Nadefinujte si správně testovací třídu s dědičností a zkontrolujte metodu pomocí nástroje `assertEqual()`.

Řešení úkolu 2

```
import unittest

# 1. Část produkčního kódu
def odecti(a, b):
    return a - b

# 2. Část testovací (zpravidla leží v samostatném souboru)
class TestMatematiky(unittest.TestCase):

    # Testovací metody musí povinně začínat slovem "test_"
    def test_odcitani(self):
        vysledek = odecti(10, 3)

        # Testovací nástroj automaticky ověří, zda 7 odpovídá výsledku
        self.assertEqual(vysledek, 7)
```

Úkol 3 (Těžká obtížnost)

Zadání úkolu: Správný test musí zajistit, že se kód nerozbije nekontrolovatelně, dostane-li neplatná data. Napište funkci `vydel(a,b)` a pomocí `unittest.TestCase` otestujte, že při pokusu o dělení nulou funkce správně zachytí tento pokus a "vyhodí" systémovou chybu (Exception) `ZeroDivisionError`.

Řešení úkolu 3

```
import unittest

def vydel(a, b):
    return a / b

class TestKrajnichPripadu(unittest.TestCase):

    def test_ochrana_proti_deleni_nulou(self):

        # self.assertRaises očekává, že následující odsazený kód skončí
chybou.
        # Výsledek tohoto testu je tak paradoxně zelený (Success),
        # jen pokud funkce skutečně kriticky selže!
        with self.assertRaises(ZeroDivisionError):
            vydel(15, 0)
```

Úkol 4 (Lehká obtížnost - dodatek)

Zadání úkolu: (Jednořádkový assert 2). Máte program, který připojuje data do pole (seznamu). Vyzkoušejte klíčové slovo `assert` ke spolehlivému ověření toho, že po přidání prvku do pole už toto pole skutečně obsahuje alespoň 1 prvek (za pomoci funkce `len`).

Řešení úkolu 4

```
fronta_na_tisk = []

# Vykonání akce
fronta_na_tisk.append("dokument1.pdf")

# Rychlá vývojářská kontrola. Pokud fronta není alespoň 1, program selže.
# Toto je typické pro Debugging (vyhledávání chyb za běhu).
assert len(fronta_na_tisk) == 1, "Chyba: Položka se nedokázala přidat do
fronty!"

print("Systém prošel rychlým testem, funguje v pořádku.")
```

Úkol 5 (Střední obtížnost - dodatek)

Zadání úkolu: Mějme produkční funkci `je_plnolety(vek)`, která má správně vracet Boolean. Napište třídu s `unittest`em a pro ověření výsledku nyní nevyužívejte `assertEqual` (rovná se hodnota), ale otestujte to vysoce specifikovanými kontrolami `assertTrue` a `assertFalse` z arzenálu modulu unittest.

Řešení úkolu 5

```
import unittest

def je_plnolety(vek):
    return vek >= 18

class TestPlnoletosti(unittest.TestCase):

    def test_dospelosti(self):
        # Nástroj kontroluje, zda funkce pro hodnotu 20 vrátí RYZE Třídu
'True'
        self.assertTrue(je_plnolety(20))

    def test_nedospelosti(self):
        # A naopak obrácená logika pro edge-cases (okrajové možnosti)
        self.assertFalse(je_plnolety(12))
```

Úkol 6 (Těžká obtížnost - dodatek)

Zadání úkolu: Představte si, že máte 5 testovacích metod (test_...) a všechny potřebují testovat např. vytvořenou instanci Banky. Místo psaní `banka = BankovniUcet()` v každém z 5 testů znovu a znovu, ukažte nasazení funkce `setUp()` z modulu unittest. Tato metoda se u dobrých testerů spouští plně automaticky před každým testem a data "předžvýká".

Řešení úkolu 6

```
import unittest

class TestSlozitychDat(unittest.TestCase):

    # Tato speciální metoda se zavolá SAMASAMO a AUTOMATICKY PŘED začátkem
každé test  metody
    def setUp(self):
        # Připravíme si sdílená čerstvá data a uložíme je do 'self'
        self.testovaci_pole = [10, 20, 30]

    def test soucet(self):
        # Nemusíme tvořit pole, čerpáme ho přímo ze setupu ze self
        self.assertEqual(sum(self.testovaci_pole), 60)

    def test delka(self):
        self.assertEqual(len(self.testovaci_pole), 3)
```